AJAY.I - Foundations of Agentic AI Codes

The easiest approach is to use Python 3 and VS Code or IDLE, with a separate `.py` file for each experiment. Most of your record's programs are educational simulations and do not require a paid LLM API or API key. Only Experiment 8 requires an additional package; Experiment 10 uses SQLite, which is built into Python. I'll preserve your record's topics and give sample outputs consistent with each program.

1. Fresh-PC setup

Install Python

1. Install Python 3 from python.org/downloads.
2. On Windows, select Add Python to PATH during installation.
3. Open Command Prompt and verify:

```
python --version
```

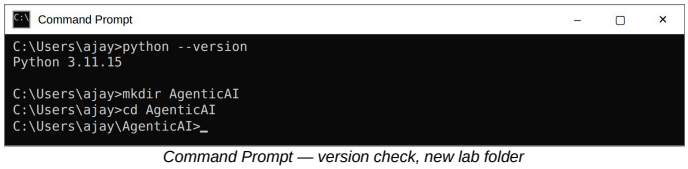
Create a lab folder

```
mkdir AgenticAI
cd AgenticAI
```

Save each experiment as `exp1.py`, `exp2.py`, and so on. Run a program using:

```
python exp1.py
```

OUTPUT



Command Prompt — version check, new lab folder

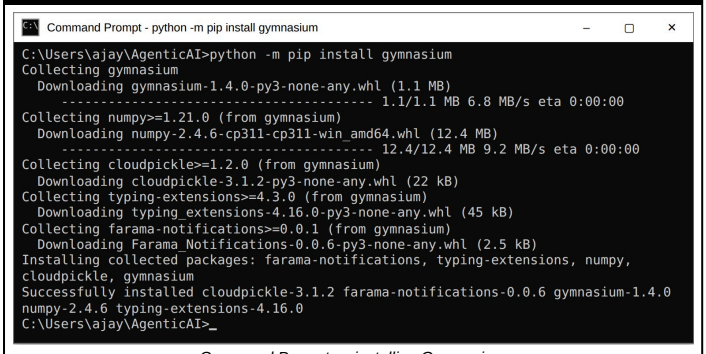
2. Packages required for each experiment

Experiment	Additional installation	Internet/API key
1 — OODA loop	None	No
2 — Thought–Action	None (`re` is built in)	No
3 — Observation cycle	None	No
4 — Tool calling	None (`re` is built in)	No
5 — Structured output	None (`json`, `re` built in)	No
6 — System 1 vs System 2	None	No
7 — CoALA	None	No
8 — Gymnasium environment	`pip install gymnasium`	Usually no
9 — ReAct, Reflection, Planning	None	No
10 — RAG and MemGPT concepts	None (`sqlite3`, `re` built in)	No

Install Gymnasium for Experiment 8 with:

```
python -m pip install gymnasium
```

OUTPUT



Command Prompt — installing Gymnasium

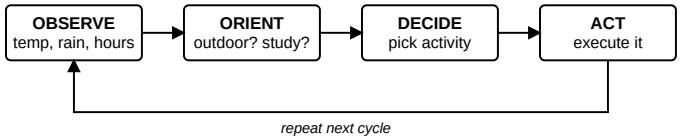
Note: These simple programs demonstrate the concepts in your record. Experiments 2, 4 and 5 are rule-based simulations rather than genuine LLM API integrations; Experiment 10 is a simplified local RAG-style retrieval and persistent-memory demonstration. Your record explicitly makes these distinctions.

Experiment 1 — Implement the OODA loop

Record topic: Observe → Orient → Decide → Act · `exp1.py`

Concept: The agent observes the weather and free time, analyses the situation, chooses an activity, and simulates the action.

FLOW



OODA: the cycle repeats for every new observation

```
def observe():
    print("\n--- OBSERVE ---")
    temp = float(input("Enter temperature (C): "))
    rain = input("Is it raining? (yes/no): ").lower() == "yes"
    hours = float(input("Enter free hours: "))
    return temp, rain, hours

def orient(temp, rain, hours):
    outdoor = not rain and temp <= 35
    long_study = hours >= 3
    print("--- ORIENT ---")
    print("Outdoor suitable:", outdoor)
    print("Long study possible:", long_study)
    return outdoor, long_study

def decide(outdoor, study):
    print("--- DECIDE ---")
    if outdoor and study:
        return "Study in library, then take a walk"
    elif outdoor:
        return "Take a short walk and revise notes"
    elif study:
        return "Stay indoors and study for three hours"
    else:
        return "Stay indoors and revise for one hour"

def act(decision):
    print("--- ACT ---")
    print("Executing:", decision)

print("=== OODA Study Assistant ===")
for cycle in range(1, 3):
    print("\nCycle", cycle)
    temp, rain, hours = observe()
    outdoor, study = orient(temp, rain, hours)
    decision = decide(outdoor, study)
    print("Decision:", decision)
    act(decision)

print("\nOODA loop completed.")
```

Sample run

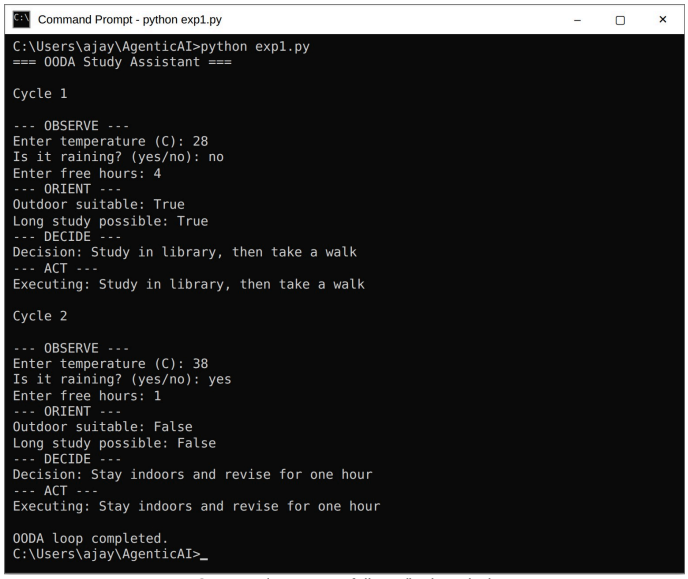
```
=== OODA Study Assistant ===

Cycle 1
--- OBSERVE ---
Enter temperature (C): 28
Is it raining? (yes/no): no
Enter free hours: 4
--- ORIENT ---
Outdoor suitable: True
Long study possible: True
--- DECIDE ---
Decision: Study in library, then take a walk
--- ACT ---
Executing: Study in library, then take a walk

Cycle 2
Enter temperature (C): 38
Is it raining? (yes/no): yes
Enter free hours: 1
Outdoor suitable: False
Long study possible: False
Decision: Stay indoors and revise for one hour
Executing: Stay indoors and revise for one hour

OODA loop completed.
```

OUTPUT



Command Prompt — full run (both cycles)

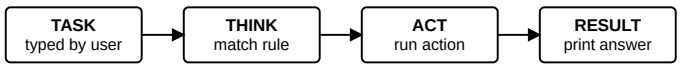
Remember: OODA = Observe, Orient, Decide, Act.

Experiment 2 — LLM-based agent using the Thought–Action cycle

Record topic: Interpret a task, choose an action, execute it · `exp2.py`

Concept: The program recognises addition, multiplication, reversing text, and counting words.

FLOW



rules: add · multiply · reverse · count words

Thought decides what to do, Action executes it

```python
import re

def think(task):
    text = task.lower()
    nums = [float(n) for n in re.findall(r"\d+\.?\d*", text)]
    if "add" in text and len(nums) >= 2:
        return "Addition requested", "add", nums
    if "multiply" in text and len(nums) >= 2:
        return "Multiplication requested", "multiply", nums
    if "reverse" in text:
        return "Reverse text", "reverse", [text.split("reverse",
1)[1].strip()]
    if "count" in text and "words" in text:
        return "Count words", "count", [task.split(":", 1)
[-1].strip()]
    return "No matching action", "none", []

def act(action, args):
    if action == "add":
        return sum(args)
    if action == "multiply":
        result = 1
        for n in args:
            result *= n
        return result
    if action == "reverse":
        return args[0][::-1]
    if action == "count":
        return len(args[0].split())
    return "Unable to complete task"

print("=== Thought-Action Agent ===")
print("Rule-based simulation; no LLM API used.")

while True:
    task = input("\nEnter task (or exit): ").strip()
    if task.lower() == "exit":
        print("Agent stopped.")
        break
    thought, action, args = think(task)
    print("Thought:", thought)
    print("Action:", action, args)
    print("Result:", act(action, args))
```

Sample output

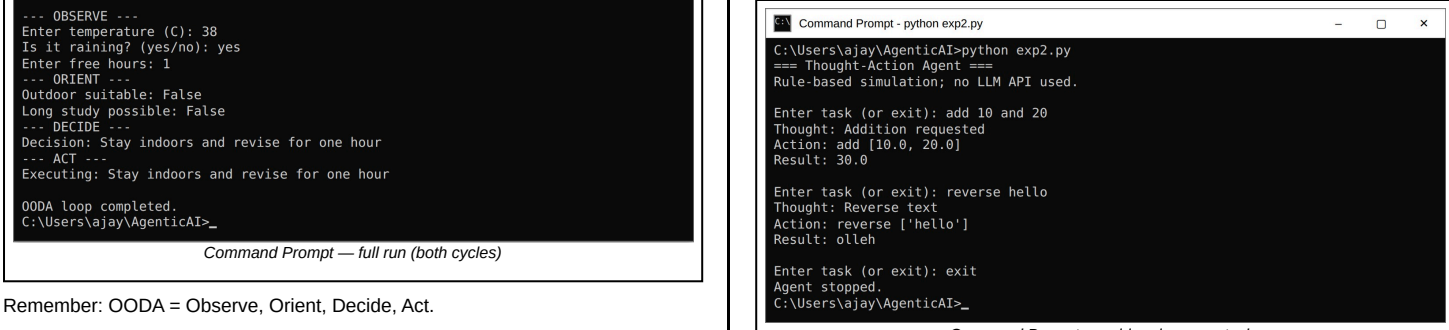
```
=== Thought-Action Agent ===
Rule-based simulation; no LLM API used.

Enter task (or exit): add 10 and 20
Thought: Addition requested
Action: add [10.0, 20.0]
Result: 30.0

Enter task (or exit): reverse hello
Thought: Reverse text
Action: reverse ['hello']
Result: olleh

Enter task (or exit): exit
Agent stopped.
```

OUTPUT



Command Prompt — add and reverse tasks

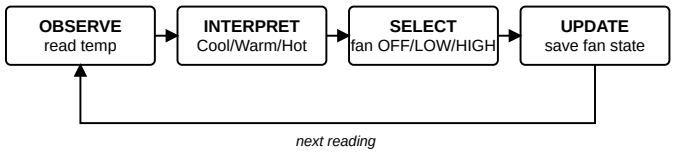
Remember: Thought decides what to do; Action executes it.

Experiment 3 — Develop an agent using the Observation cycle

Record topic: Temperature-monitoring agent · `exp3.py`

Concept: The agent repeatedly observes temperature and decides whether to turn the fan on or off.

FLOW



Observation cycle for each temperature reading

```python
readings = [24, 28, 33, 38, 31, 22]
fan = "OFF"

print("=== Temperature Monitoring Agent ===")

for i, temp in enumerate(readings, 1):
    print(f"Cycle {i}: Temperature = {temp} C")

    if temp < 26:
        state = "Cool"
    elif temp < 32:
        state = "Warm"
    else:
        state = "Hot"

    if state == "Hot":
        action = "Turn fan ON at HIGH speed"
        fan = "HIGH"
    elif state == "Warm" and fan == "OFF":
        action = "Turn fan ON at LOW speed"
        fan = "LOW"
    elif state == "Cool" and fan != "OFF":
        action = "Turn fan OFF"
        fan = "OFF"
    else:
        action = "No change"

    print("Interpretation:", state)
    print("Action:", action)
    print("Fan state:", fan)
    print()

print("Monitoring completed.")
```

Sample output

```
=== Temperature Monitoring Agent ===
Cycle 1: Temperature = 24 C
Interpretation: Cool
Action: No change
Fan state: OFF

Cycle 2: Temperature = 28 C
Interpretation: Warm
```

```
Action: Turn fan ON at LOW speed
Fan state: LOW

Cycle 3: Temperature = 33 C
Interpretation: Hot
Action: Turn fan ON at HIGH speed
Fan state: HIGH

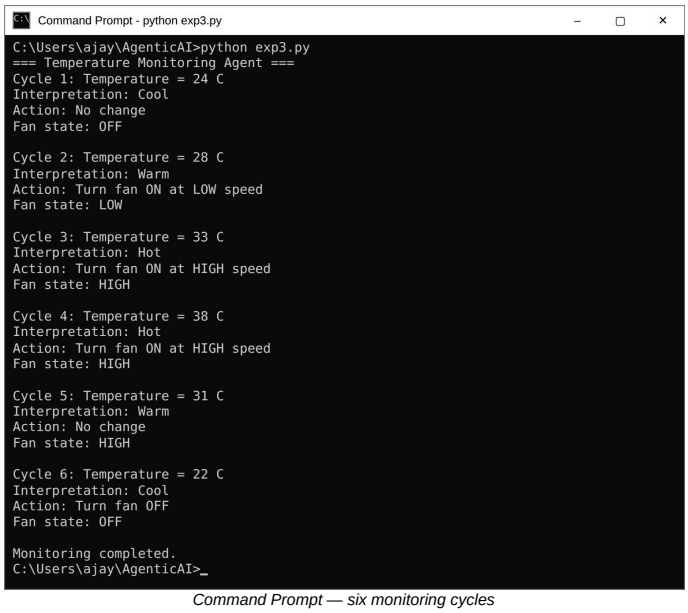
Cycle 4: Temperature = 38 C
Interpretation: Hot
Action: Turn fan ON at HIGH speed
Fan state: HIGH

Cycle 5: Temperature = 31 C
Interpretation: Warm
Action: No change
Fan state: HIGH

Cycle 6: Temperature = 22 C
Interpretation: Cool
Action: Turn fan OFF
Fan state: OFF

Monitoring completed.
```

OUTPUT



Command Prompt — six monitoring cycles

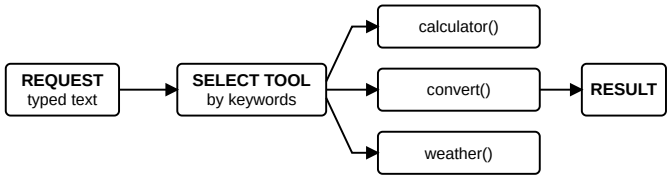
Remember: Observe → Interpret → Select action → Update state.

Experiment 4 — Implement tool calling

Record topic: Calculator, unit converter and weather lookup · `exp4.py`

Concept: The agent selects a Python function based on the user's request. The weather values are sample data, not live weather.

FLOW



The agent picks one Python function per request

```python
import re

def calculator(a, b, op):
    if op == "+": return a + b
    if op == "-": return a - b
    if op == "*": return a * b
    if op == "/": return a / b if b != 0 else "Division by zero"

def convert(value, unit):
    if unit == "km":
        return f"{value * 0.621371:.2f} miles"
    if unit == "kg":
        return f"{value * 2.20462:.2f} pounds"
    if unit == "c":
        return f"{value * 9 / 5 + 32:.1f} F"

def weather(city):
    data = {
        "chennai": "32 C, humid",
        "coimbatore": "27 C, cloudy"
    }
    return data.get(city, "No data available")

print("=== Tool Calling Agent ===")

while True:
    request = input("\nRequest (or exit): ").lower().strip()
    if request == "exit":
        print("Agent stopped.")
        break

    if "convert" in request:
        m = re.search(r"(\d+\.?\d*)\s*(km|kg|c)\b", request)
        print("Tool selected: unit converter")
        if m:
            print("Result:", convert(float(m.group(1)),
m.group(2)))
        else:
            print("Result: Invalid conversion request")
    elif "weather" in request:
        city = request.split()[-1]
        print("Tool selected: weather")
        print("Result:", weather(city))
    else:
        m = re.search(r"(\d+\.?\d*)\s*([+\-*/])\s*(\d+\.?\d*)",
request)
        if m:
            print("Tool selected: calculator")
            print("Result:", calculator(float(m.group(1)),
                                        float(m.group(3)),
m.group(2)))
        else:
            print("No suitable tool found")
```

Sample output

```
=== Tool Calling Agent ===

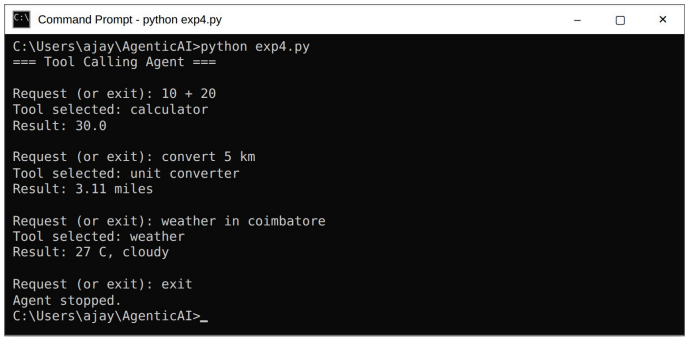
Request (or exit): 10 + 20
Tool selected: calculator
Result: 30.0

Request (or exit): convert 5 km
Tool selected: unit converter
Result: 3.11 miles

Request (or exit): weather in coimbatore
Tool selected: weather
Result: 27 C, cloudy

Request (or exit): exit
Agent stopped.
```

OUTPUT



Command Prompt — calculator, converter and weather tools

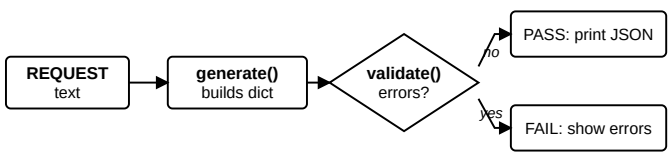
Remember: Select the tool → pass arguments → execute → return result.

Experiment 5 — Structured output generation

Record topic: Generate and validate JSON · `exp5.py`

Concept: Convert a study request into structured JSON and check its fields.

FLOW



generate() builds a dict, validate() checks it

```python
import json
import re

def generate(request):
    text = request.lower()
    priority = "medium"
    for level in ("low", "medium", "high"):
        if level in text:
            priority = level
```

```
    match = re.search(r"(\d+\.?\d*)\s*hours?", text)
    duration = float(match.group(1)) if match else None
    task = re.split(r",| for \d", request)[0].strip()
    return {
        "task": task,
        "priority": priority,
        "duration_hours": duration,
        "status": "pending"
    }

def validate(data):
    errors = []
    if not data["task"]:
        errors.append("Task cannot be empty")
    if data["priority"] not in ("low", "medium", "high"):
        errors.append("Invalid priority")
    if data["duration_hours"] is None or data["duration_hours"] <=
0:
        errors.append("Duration must be positive")
    return errors

print("=== Structured Output Generator ===")

while True:
    request = input("\nStudy request (or exit): ").strip()
    if request.lower() == "exit":
        print("Generator stopped.")
        break
    data = generate(request)
    errors = validate(data)
    if errors:
        print("Validation FAILED:", errors)
    else:
        print("Validation PASSED")
        print(json.dumps(data, indent=2))
```

Sample output

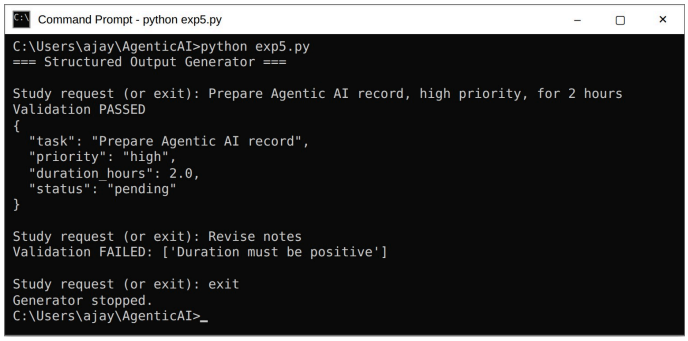
```
=== Structured Output Generator ===

Study request (or exit): Prepare Agentic AI record, high priority,
for 2 hours
Validation PASSED
{
  "task": "Prepare Agentic AI record",
  "priority": "high",
  "duration_hours": 2.0,
  "status": "pending"
}

Study request (or exit): Revise notes
Validation FAILED: ['Duration must be positive']

Study request (or exit): exit
Generator stopped.
```

OUTPUT



Command Prompt — one valid and one invalid request

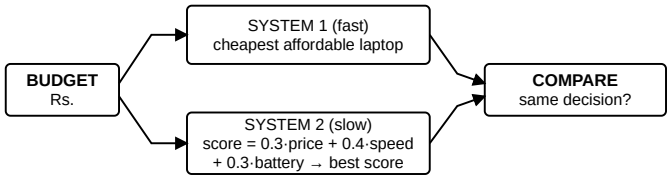
Remember: JSON represents structured data; validation checks whether the fields are acceptable.

Experiment 6 — Compare System 1 and System 2 reasoning

Record topic: Select a laptop within a budget · `exp6.py`

Concept: System 1 quickly selects the cheapest affordable laptop. System 2 calculates a weighted score using price, speed and battery life.

FLOW



Fast rule vs weighted analysis

```
laptops = [
    {"name": "Laptop A", "price": 38000, "speed": 5, "battery":
6},
    {"name": "Laptop B", "price": 55000, "speed": 8, "battery":
9},
    {"name": "Laptop C", "price": 72000, "speed": 9, "battery":
7},
    {"name": "Laptop D", "price": 45000, "speed": 7, "battery": 5}
]

def system1(budget):
    affordable = [x for x in laptops if x["price"] <= budget]
    return min(affordable, key=lambda x: x["price"])

def score(x, budget):
    return (0.3 * ((budget - x["price"]) / budget * 10)
            + 0.4 * x["speed"] + 0.3 * x["battery"])

def system2(budget):
    affordable = [x for x in laptops if x["price"] <= budget]
    return max(affordable, key=lambda x:
        0.3 * ((budget - x["price"]) / budget * 10)
        + 0.4 * x["speed"] + 0.3 * x["battery"])

print("=== System 1 vs System 2 ===")

for budget in (40000, 60000, 80000):
    fast = system1(budget)
    deep = system2(budget)
    print("\nBudget: Rs.", budget)
    print("System 1:", fast["name"])
    print("System 2:", deep["name"], "Score:", round(score(deep,
budget), 2))
    print("Same decision:", fast["name"] == deep["name"])
```

Sample output

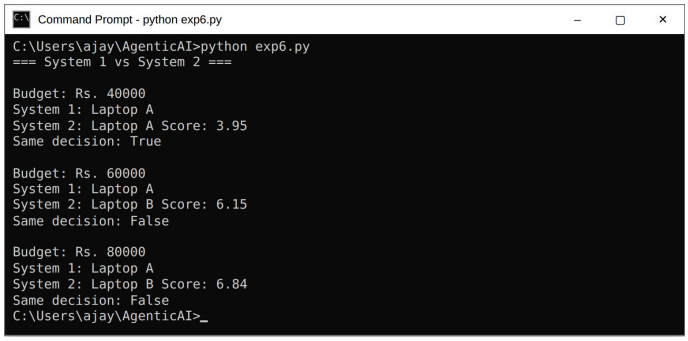
```
=== System 1 vs System 2 ===

Budget: Rs. 40000
System 1: Laptop A
System 2: Laptop A Score: 3.95
Same decision: True

Budget: Rs. 60000
System 1: Laptop A
System 2: Laptop B Score: 6.15
Same decision: False

Budget: Rs. 80000
System 1: Laptop A
System 2: Laptop B Score: 6.84
Same decision: False
```

OUTPUT



Command Prompt — three budgets

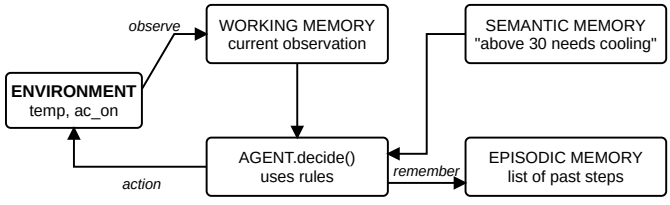
Remember: System 1 = fast rule. System 2 = deeper, multi-factor analysis.

Experiment 7 — CoALA-based cognitive architecture

Record topic: Working, episodic and semantic memory · `exp7.py`

Concept: The agent observes the temperature, uses stored knowledge to decide when to operate an AC, and records past actions.

FLOW



Memory types used by the CoALA agent

```
class Environment:
    def __init__(self):
        self.temperature = 28
        self.ac_on = False

    def observe(self):
        return {"temperature": self.temperature, "ac_on":
self.ac_on}

    def execute(self, action):
        if action == "turn_on_ac":
            self.ac_on = True
        elif action == "turn_off_ac":
            self.ac_on = False
        self.temperature += -2 if self.ac_on else 3
        return self.temperature

class Agent:
    def __init__(self):
        self.working = {}
        self.episodic = []
        self.semantic = "Temperature above 30 needs cooling"

    def decide(self, obs):
        self.working = dict(obs)
        temp = obs["temperature"]
        if temp > 30:
            print("Knowledge:", self.semantic)
        if temp > 30 and not obs["ac_on"]:
            return "turn_on_ac"
        if temp < 24 and obs["ac_on"]:
            return "turn_off_ac"
        return "wait"

    def remember(self, obs, action, new_temp):
        self.episodic.append((obs, action, new_temp))

env = Environment()
agent = Agent()

print("=== CoALA Cognitive Agent ===")

for step in range(1, 10):
    obs = env.observe()
    print("Step:", step, "| Observation:", obs)
    action = agent.decide(obs)
    new_temp = env.execute(action)
    agent.remember(obs, action, new_temp)
    print("Action:", action, "| New temperature:", new_temp)
    print()

print("Episodic memory:", len(agent.episodic), "episodes")
```

Sample output (beginning and ending)

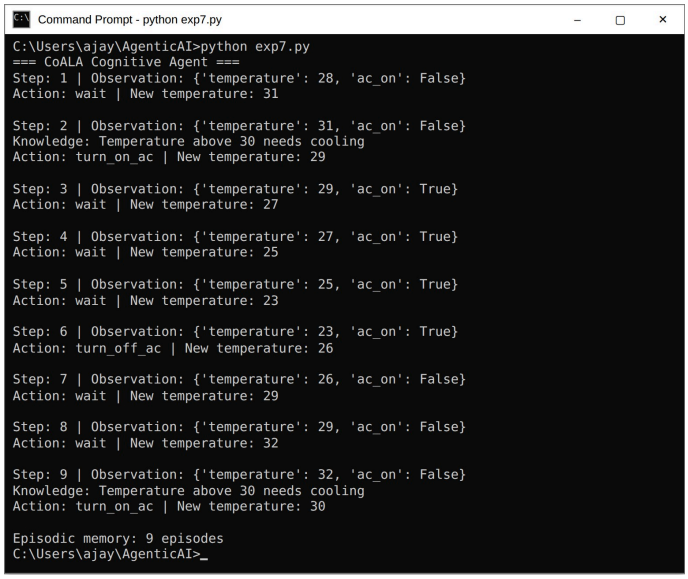
```
=== CoALA Cognitive Agent ===
Step: 1 | Observation: {'temperature': 28, 'ac_on': False}
Action: wait | New temperature: 31

Step: 2 | Observation: {'temperature': 31, 'ac_on': False}
Knowledge: Temperature above 30 needs cooling
Action: turn_on_ac | New temperature: 29

...
Step: 9 | Observation: {'temperature': 32, 'ac_on': False}
Knowledge: Temperature above 30 needs cooling
Action: turn_on_ac | New temperature: 30

Episodic memory: 9 episodes
```

OUTPUT



Command Prompt — all nine steps

Remember: CoALA = cognitive architecture with working memory, episodic memory, semantic knowledge and action selection.

Experiment 8 — Gymnasium-based intelligent agent environment

Record topic: Grid-world environment · `exp8.py`

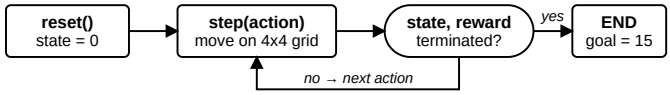
Additional package: `gymnasium`

Install it before running:

```
python -m pip install gymnasium
```

Concept: The agent moves through a 4×4 grid using actions Up, Down, Left and Right. Reaching the goal gives a reward of `10`.

FLOW



reset() starts an episode, step() returns the new state, reward and done flag

```
import gymnasium as gym
from gymnasium import spaces

class GridWorldEnv(gym.Env):
    def __init__(self):
        super().__init__()
        self.observation_space = spaces.Discrete(16)
        self.action_space = spaces.Discrete(4)
        self.state = 0
        self.steps = 0

    def reset(self, seed=None, options=None):
        super().reset(seed=seed)
        self.state = 0
        self.steps = 0
        return self.state, {}

    def step(self, action):
        row, col = divmod(self.state, 4)
        if action == 0:
            row = max(row - 1, 0)
        elif action == 1:
            row = min(row + 1, 3)
        elif action == 2:
            col = max(col - 1, 0)
        else:
            col = min(col + 1, 3)
        self.state = row * 4 + col
        self.steps += 1
        terminated = self.state == 15
        reward = 10.0 if terminated else -0.1
        return self.state, reward, terminated, False, {}

env = GridWorldEnv()
names = {0: "Up", 1: "Down", 2: "Left", 3: "Right"}
state, _ = env.reset()
print("Observation space:", env.observation_space)
print("Action space:", env.action_space)
print("Start state:", state)

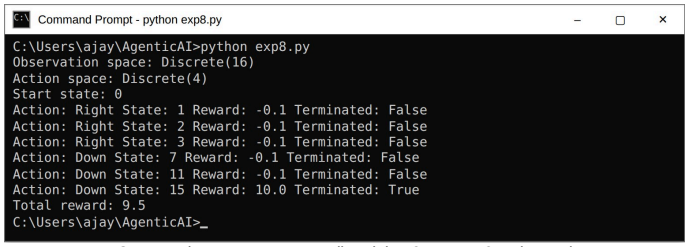
total = 0
for action in [3, 3, 3, 1, 1, 1]:
    state, reward, terminated, truncated, _ = env.step(action)
    total += reward
    print("Action:", names[action], "State:", state,
          "Reward:", reward, "Terminated:", terminated)
    if terminated:
        break

print("Total reward:", round(total, 2))
```

Sample output

```
Observation space: Discrete(16)
Action space: Discrete(4)
Start state: 0
Action: Right State: 1 Reward: -0.1 Terminated: False
Action: Right State: 2 Reward: -0.1 Terminated: False
Action: Right State: 3 Reward: -0.1 Terminated: False
Action: Down State: 7 Reward: -0.1 Terminated: False
Action: Down State: 11 Reward: -0.1 Terminated: False
Action: Down State: 15 Reward: 10.0 Terminated: True
Total reward: 9.5
```

OUTPUT



Command Prompt — agent walks Right ×3, Down ×3 to the goal

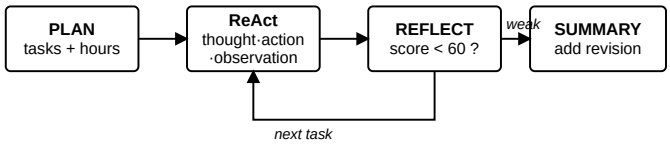
Remember: `reset()` starts the environment; `step()` executes an action and returns the state, reward and termination information.

Experiment 9 — ReAct, Reflection and Planning

Record topic: Automated study planner · `exp9.py`

Concept: The agent plans study tasks, performs simulated actions, evaluates test scores, and adds revision if a score is below 60%.

FLOW



Plan the tasks, act on each, reflect on the score

```
scores = {
    "Revise notes": 85,
    "Practice programs": 55,
    "Mock test": 70
}

def plan(hours):
    tasks = [("Revise notes", 2),
             ("Practice programs", 3),
             ("Mock test", 2)]
    if sum(h for _, h in tasks) > hours:
        tasks = [(name, max(1, h - 1)) for name, h in tasks]
    print("PLANNING:", tasks)
    return tasks

def react(task, hours):
    print("\nTask:", task)
    print("Thought: I need", hours, "hours.")
    print("Action: Start", task)
    result = scores[task]
    print("Observation: Score =", result)
    return result

def reflect(task, score):
    if score < 60:
        print("Reflection: Low score. Revision needed.")
        return task
    print("Reflection: Score acceptable.")
    return None

print("=== ReAct, Reflection and Planning ===")
hours = int(input("Enter available hours: "))
revise = []

for task, h in plan(hours):
    score = react(task, h)
    weak = reflect(task, score)
    if weak:
        revise.append(weak)

print("\nREFLECTION SUMMARY")
if revise:
    print("Add revision for:", ", ".join(revise))
else:
    print("No extra revision needed")
print("Study plan completed.")
```

Sample output (enter `4` hours)

```
=== ReAct, Reflection and Planning ===
Enter available hours: 4
PLANNING: [('Revise notes', 1),
 ('Practice programs', 2), ('Mock test', 1)]

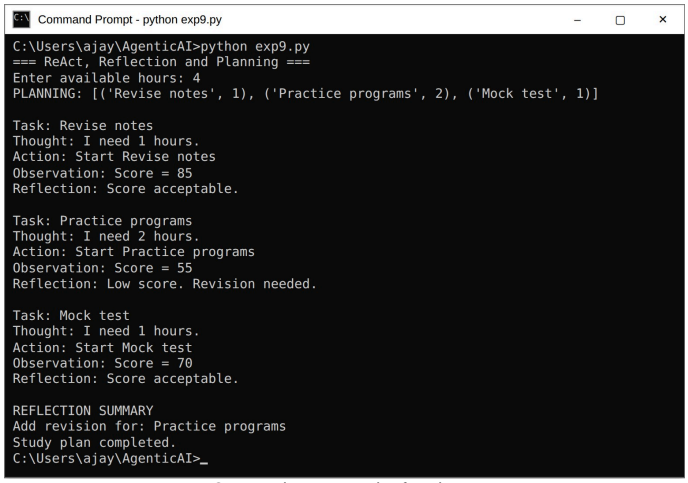
Task: Revise notes
Thought: I need 1 hours.
Action: Start Revise notes
Observation: Score = 85
Reflection: Score acceptable.

Task: Practice programs
Thought: I need 2 hours.
Action: Start Practice programs
Observation: Score = 55
Reflection: Low score. Revision needed.

Task: Mock test
Thought: I need 1 hours.
Action: Start Mock test
Observation: Score = 70
Reflection: Score acceptable.

REFLECTION SUMMARY
Add revision for: Practice programs
Study plan completed.
```

OUTPUT



Command Prompt — plan for 4 hours

Remember: Planning creates tasks; ReAct performs them; Reflection checks the result and improves the plan.

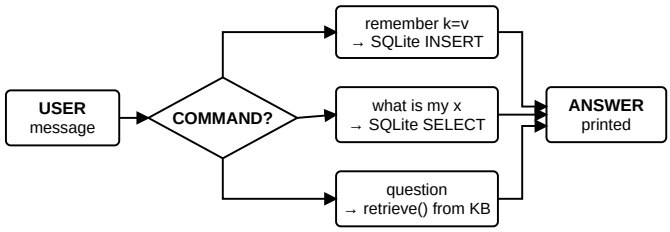
Experiment 10 — Memory-enabled agent using RAG and MemGPT concepts

Record topic: Knowledge retrieval and persistent memory · `exp10.py`

Requirements: Python's built-in `sqlite3` and `re` modules. No additional packages or API keys.

Concept: The agent retrieves relevant information from a small knowledge base and remembers user preferences in an SQLite database. Its memory remains after restarting the program.

FLOW



Messages are routed to memory commands or to knowledge retrieval

```
import re
import sqlite3

knowledge = [
    ("RAG", "RAG retrieves documents and uses them to answer
questions."),
    ("MemGPT", "MemGPT-style agents use core and archival
memory."),
    ("Tool calling", "Tool calling selects a function and passes
arguments.")
]

db = sqlite3.connect("rag_memory.db")
db.execute("CREATE TABLE IF NOT EXISTS memory "
           "(key TEXT PRIMARY KEY, value TEXT)")
db.commit()

def remember(key, value):
    db.execute("INSERT OR REPLACE INTO memory VALUES (?, ?)",
               (key, value))
    db.commit()

def recall(key):
    row = db.execute("SELECT value FROM memory WHERE key=?",
                     (key,)).fetchone()
    return row[0] if row else None

def retrieve(question):
    q = question.lower()
```

```
    for title, text in knowledge:
        if title.lower() in q:
            return f"[Retrieved: {title}] {text}"
    words = set(re.findall(r"\w+", q))
    for title, text in knowledge:
        if words & set(re.findall(r"\w+", text.lower())) - {"and",
"the", "a"}:
            return f"[Retrieved: {title}] {text}"
    return "No relevant knowledge found."

print("=== RAG Agent with Memory ===")
print("Stored style:", recall("style"))
print("Stored topic:", recall("topic"))

while True:
    msg = input("\nYou: ").strip()
    low = msg.lower()
    if low == "exit":
        print("Session ended. Memory saved.")
        break
    if low.startswith("remember "):
        key, _, value = msg[9:].partition("=")
        remember(key.strip(), value.strip())
        print("Agent: Saved", key.strip(), "=", value.strip())
    elif low.startswith("what is my "):
        print("Agent:", recall(low[11:].strip()) or "I do not know
yet.")
    else:
        print("Agent:", retrieve(msg))
```

Sample output — Session 1

```
=== RAG Agent with Memory ===
Stored style: None
Stored topic: None

You: remember style=short
Agent: Saved style = short

You: remember topic=RAG
Agent: Saved topic = RAG

You: what is RAG
Agent: [Retrieved: RAG] RAG retrieves documents and uses them to
answer questions.

You: exit
Session ended. Memory saved.
```

Sample output — Session 2, after restarting the program

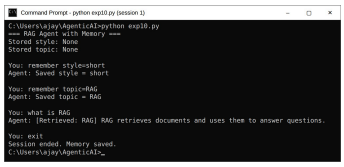
```
=== RAG Agent with Memory ===
Stored style: short
Stored topic: RAG

You: what is my style
Agent: short

You: explain MemGPT
Agent: [Retrieved: MemGPT] MemGPT-style agents use core and
archival memory.

You: exit
Session ended. Memory saved.
```

OUTPUT



Session 1 — preferences saved



Session 2 — after restarting, memory remains

Remember: RAG retrieves relevant knowledge; SQLite stores persistent memory. This is a simplified educational implementation, not the official MemGPT framework.

Quick revision before your practical

Exp.	Main concept	One-line memory trick
1	OODA	Observe → Orient → Decide → Act
2	Thought–Action	Think, select an action, execute
3	Observation cycle	Observe the temperature and update the fan
4	Tool calling	Select and execute a Python function
5	Structured output	Generate and validate JSON
6	System 1 vs 2	Fast rule vs weighted analysis
7	CoALA	Working, episodic and semantic memory
8	Gymnasium	State, action, reward and termination
9	ReAct/Reflection/Planning	Plan, act, observe and improve
10	RAG/MemGPT	Retrieve knowledge and remember preferences

Important correction for Experiment 7: The code above's exact final temperature depends on the rules executed each cycle. With the given rules, the ninth step ends at 30°C with the AC ON; the sample ending in the earlier section should not be copied as exact output.

All programs can be saved and run independently. For your exam, practise typing the code, explaining the main concept, and demonstrating the expected output.